

Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 8

Recap and outline

- Re-visiting MDI
- Wrapper methods
- Optimization challenges
- Is re-training necessary?
- Slight tangent to pseudo-inverse for least-squares

Recall MDI

- MDI (mean decrease in impurity)
- How much does each feature decrease impurity (makes split more pure)
- Embedded since MDI is used to compute each decision tree in ensemble
- We take an average over all the trees

Is MDI a good algorithm?

- **Statistical:** How close is $\widehat{S}$ to S^* ?
- **Computational:** Can we find $\widehat{S}$ in reasonable time?
- Computationally, not too expensive given that it is embedded (no more expensive than prediction)
- Is it statistically reliable?

Example

$$Y = X_1 \oplus X_2$$

where $X_1 \sim \text{Bernoulli}(1/2)$, $X_2 \sim \text{Bernoulli}(1/2)$ independent.

- If $(X_1, X_2) = (1, 1)$ or $(0, 0)$, $Y = 0$
- If $(X_1, X_2) = (1, 0)$ or $(0, 1)$, $Y = 1$
- If $X_1 = 0$, $Y = X_2$ and $X_1 = 1$, $Y = 1 - X_2$ (and vice-versa)
- Both features significant: $Y \not\perp X_1|X_2$ and $Y \not\perp X_2|X_1$

Recall, for feature selection:

$$\tilde{S}^* := \{j \subset \{1, 2, \dots, p\} : Y \not\perp X_j|X_{-j}\} = \{1, 2\}$$

What does MDI do?

- Recall that MDI first decides whether to split along X_1 or X_2 based on which one decreases impurity
- Let's use Gini:

$$1 - \sum_{k=1}^K p_k^2$$

- Overall Gini for Y is $0.5 (1 - 0.5^2 - 0.5^2)$

What does MDI do for this example?

- If we split along X_1
- If $X_1 = 1$, Gini is $0.5 (1 - 0.5^2 - 0.5^2)$
- If $X_1 = 0$, Gini is $0.5 (1 - 0.5^2 - 0.5^2)$
- So the mean impurity is still 0.5
- So MDI is 0
- We get exactly the same result for X_2

What is going on in this example?

- MDI is 0 for both features
- Why does this happen?

What is going on in this example?

- MDI is 0 for both features
- Why does this happen?
- $Y \perp X_1$ and $Y \perp X_2$
- We have a case where $Y \perp X_1$ and $Y \perp X_2$ but $Y \not\perp X_1|X_2$ and $Y \not\perp X_2|X_1$
- Points to an issue with MDI

Shortcomings of MDI

- For each decision tree, each decision conditions only on what has come before
- Initial steps behave more like marginal dependence rather than conditional dependence
- This does not even get to issues with error due to random sampling of samples/features
- So MDI fast but not always very reliable

Recall: General feature selection formulation

General feature selection:

$$S^* \in \arg \min_{S \subseteq \{1,2,\dots,p\}, |S| \leq k} \min_{f \in \mathcal{F}} \mathbb{E}[\mathcal{L}(Y, f(X_S))]$$

Feature selection for linear regression:

$$S^* \in \arg \min_{S \subseteq \{X_1, X_2, \dots, X_p\}, |S| \leq s} \min_{\beta_S} \mathbb{E}[(Y - \sum_{j \in S} X_j \beta_j)^2]$$

Empirical (estimated quantities):

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1,2,\dots,p\}, |S| \leq k} \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, f(X_S^{(i)}))$$

Linear regression:

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1,2,\dots,p\}, |S| \leq s} \min_{\beta_S} \frac{1}{n} \sum_{i=1}^n (Y^{(i)} - \sum_{j \in S} X_j^{(i)} \beta_j)^2$$

Wrapper vs Filter vs Embedded

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1,2,\dots,p\}, |S| \leq k} \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, f(X_S^{(i)}))$$

- We have two optimizations to solve, one over subsets of features, and the other over prediction model
- Filter methods (e.g. correlation, mutual information) essentially ignore the prediction algorithm part of the optimization
- For embedded methods (e.g. Regression + p-values, Lasso, MDI), the optimization over subsets comes for free
- Wrapper methods basically get as close to original formulation as possible (do both optimizations)

Wrapper methods summarized

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1, 2, \dots, p\}, |S| \leq k} \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, f(X_S^{(i)}))$$

- For wrapper methods, given a class of prediction models/functions f , for each $S \subset \{1, 2, \dots, p\}$ we solve find the size of the objective
- Applies to any function class $\mathcal{F}$
- Huge computational challenge, need to evaluate every loss function $\mathcal{L}(., .)$ for 2^p or $\binom{p}{s}$ subsets

How do we overcome this computational challenge?

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1,2,\dots,p\}, |S| \leq k} \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, f(X_S^{(i)}))$$

- Evaluate each feature individually (reduction to p computations)
- Traverse the space of $\binom{p}{s}$ or 2^p in clever ways
- Start with an empty set and *greedily* add features (forward selection)
- Start with a full subset and remove features (feature elimination/backward selection)

Question: Do we need to re-fit/re-train the function f each time we change the subset S ?

Simple motivating example

Recall simple 2-feature non-identifiable example:

$$Y = X_1 + \frac{1}{2}X_2$$

but the features are perfectly correlated:

$$X_2 = 2X_1$$

If you fit a linear model

$$Y = \beta_1 X_1 + \beta_2 X_2,$$

any solution (β_1, β_2) of the form

$$\beta_1 + 2\beta_2 = 2$$

works.

Slight tangent: what does OLS do?

Least-squares objective:

$$\beta \in \arg \min_{\beta} \|Y - X\beta\|_2^2$$

Normal equations (linear):

$$X^T X \beta = X^T Y$$

Any (β_1, β_2) of the following form minimizes objective at 0:

$$\beta_1 + 2\beta_2 = 2$$

Can OLS get a unique solution out of this even though there are multiple solutions?

Turns out yes, but how?

- We use the concept of the pseudo or Moore-Penrose inverse

For any linear system, we can apply the pseudo-inverse to solve:

$$\begin{aligned} A\beta &= c \\ \beta &= A^\dagger c \end{aligned}$$

where $A^\dagger$ is the pseudo-inverse of A (if A is invertible then $A^\dagger = A^{-1}$ but $A^\dagger$ does not require invertibility)

Returning to least-squares and normal equations?

Least-squares objective leads to normal equations:

$$X^T X \beta = X^T Y$$

Based on pseudo-inverse,

$$\beta = (X^T X)^{\dagger} X^T Y$$

which returns a unique solution for β .

What is $(X^T X)^\dagger$

- Recall that $X^T X \in \mathbb{R}^{p \times p}$ is a positive semi-definite matrix with some rank r .
- $X^T X$ has a unique eigen-value decomposition

$$X^T X = V \Lambda V^T$$

where $V \in \mathbb{R}^{p \times r}$ where $V^T V = I_{r \times r}$ (orthogonal basis) and $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_r)$ is a diagonal matrix of eigenvalues where $\lambda_k > 0$ for all $1 \leq k \leq r$

- Using properties of eigenvalues/eigenvectors, what is a logical way to "invert" this matrix

$$(X^T X)^\dagger = V \Lambda^{-1} V^T$$

and note

$$(X^T X)^\dagger X^T X = X^T X (X^T X)^\dagger = V V^T$$

(not strictly an inverse)

So to solve least-squares

Normal equations:

$$X^T X \beta = X^T Y$$

and using pseudo-inverse

$$\beta = (X^T X)^{\dagger} X^T Y = V \Lambda^{-1} V^T X^T Y$$

we get unique solution

How are we magically removing identifiability issue?

- It can be shown that

$$A^\dagger c = \arg \min_{\beta: \|\beta\|_2, A\beta=c}$$

- Intuition is that when a matrix A is rank-deficient, it has a non-trivial null-space (along with full-rank column space)
- Adding any vector in the null-space does not change the solution
- Pseudo-inverse chooses to add 0 to the unique column space solution (clearly in the null-space) and has 0 ℓ_2 -norm which leads to the minimum ℓ_2 -norm solution

$$\beta = (X^T X)^\dagger X^T Y = V \Lambda^{-1} V^T X^T Y$$

solves the optimization problem:

$$\beta = \arg \min_{\beta: \|\beta\|_2, X^T X \beta = X^T Y}$$

Returning to example

$$Y = X_1 + \frac{1}{2}X_2$$

but the features are perfectly correlated:

$$X_2 = 2X_1$$

If you fit a linear model

$$Y = \beta_1 X_1 + \beta_2 X_2,$$

any solution (β_1, β_2) of the form

$$\beta_1 + 2\beta_2 = 2$$

works.

By using OLS solvers, we will get the minimum ℓ_2 -norm solution:

$$(\beta_1, \beta_2) = (2/5, 4/5)$$

Minimum norm solutions

- Naturally arise in under-determined linear systems
- Also naturally arise in neural network algorithms
- Neural networks have numerous minimum with zero training error
- Minimum norm (often ℓ_2) a natural way to choose solution
- Also very connected to gradient descent and other iterative approaches

Do we need to re-train for each subset?

$$Y = X_1 + \frac{1}{2}X_2$$

but the features are perfectly correlated:

$$X_2 = 2X_1$$

If you fit a linear model

$$Y = \beta_1 X_1 + \beta_2 X_2,$$

- One standard approach for wrapper methods is to begin with full subset and remove features.
- Let's see what happens when we do and don't re-train

$$S^* \in \arg \min_{S \subseteq \{X_1, X_2, \}} \min_{\beta_S} (Y - \sum_{j \in S} X_j \beta_j)^2$$

First let's not re-train

$$S^* \in \arg \min_{S \subseteq \{X_1, X_2\}} \min_{\beta_S} (Y - \sum_{j \in S} X_j \beta_j)^2$$

Note:

$$Y = X_1 + \frac{1}{2}X_2 = X_1 + \frac{1}{2}(2X_1) = 2X_1$$

$$Y = X_1 + \frac{1}{2}X_2 = \frac{1}{2}X_1 + \frac{1}{2}X_2 = X_2$$

- We already have the learned co-efficients $(\beta_1, \beta_2) = (2/5, 4/5)$
- If we drop X_1 ,

$$(Y - \beta_2 X_2)^2 = (Y - 0.8X_2)^2 > (Y - X_2)^2 = 0$$

- If we drop X_2 ,

$$(Y - \beta_1 X_1)^2 = (Y - 0.4X_1)^2 > (Y - 2X_1)^2 = 0$$

- So we would not drop either feature and the final learned subset would be (X_1, X_2)

Now let's re-train the co-efficients

Note:

$$Y = X_1 + \frac{1}{2}X_2 = X_1 + \frac{1}{2}(2X_1) = 2X_1$$

$$Y = X_1 + \frac{1}{2}X_2 = \frac{1}{2}X_1 + \frac{1}{2}X_2 = X_2$$

- If we drop X_1 ,

$$\beta_2 = \arg \min_{\beta_2} (Y - \beta_2 X_2)^2$$

- And $\beta_2 = 1$ and we'd end up with 0 error and drop X_1
- Similarly, re-training would lead to $\beta_1 = 2$ and we would drop X_2
- So either way we would drop a feature from S

Answer: Re-training is critical

For wrapper methods, if we truly want to solve:

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1, 2, \dots, p\}, |S| \leq k} \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, f(X_S^{(i)})),$$

need to re-train for each different choice of S (whether it is linear model, tree ensemble, neural network, e.t.c.)

New computational challenge: Need to run the prediction algorithm each time we change the feature subset